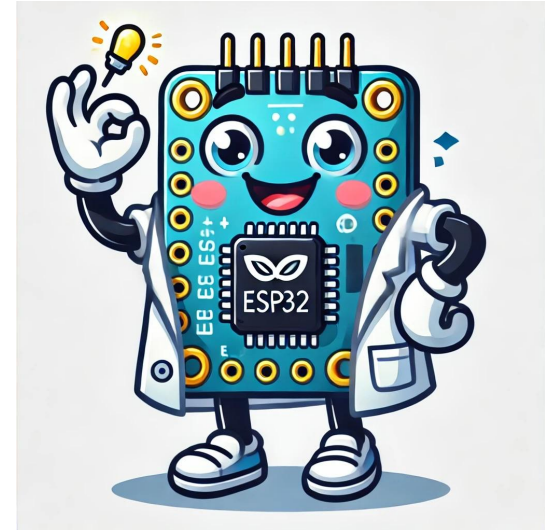




UNIVERSITÀ
degli STUDI
di CATANIA

ESP32: from Zero to Hero



davide.patti@dieei.unict.it

ESP32 Family



Advantages of ESP32



Powerful Performance

- Dual-core processor up to 240 MHz
- 520 KB SRAM & up to 4 MB Flash



Integrated Connectivity

- Built-in Wi-Fi for IoT projects
- Bluetooth v4.2 & BLE support



Versatility

- Supports GPIO, PWM, I2C, SPI, UART
- Wide sensor & actuator compatibility



Strong Ecosystem

- Arduino, PlatformIO & ESP-IDF
- Massive libraries & community support

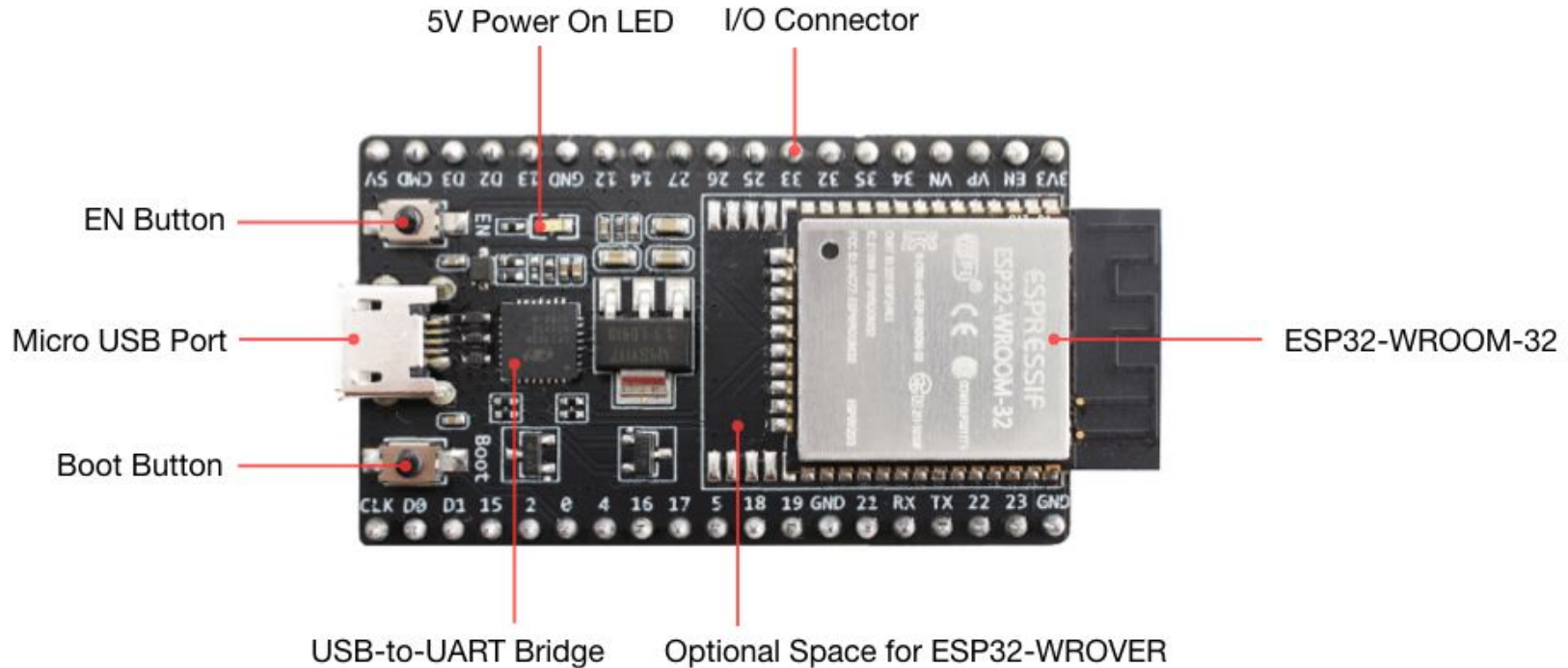
Resources

- <https://esp32io.com/>
- <https://wokwi.com/esp32>
- <https://deepbluembedded.com/getting-started-with-esp32-programming-tutorials/>
- <http://esp32.net/>

ESP32-DevKitC-32E Board

- It's very common variant, based on the ESP32-WROOM-3 module
- Being part of the ESP32 microcontroller family, all the source code examples and hardware shown in this slide will be compatible with any other ESP32





- **Reset (EN) button** is equivalent to "turning the ESP32 off and on" without physically disconnecting the power
- On most ESP32-DevKitC boards, the **boot button** is typically not required when using the Arduino IDE because of auto-reset functionality.

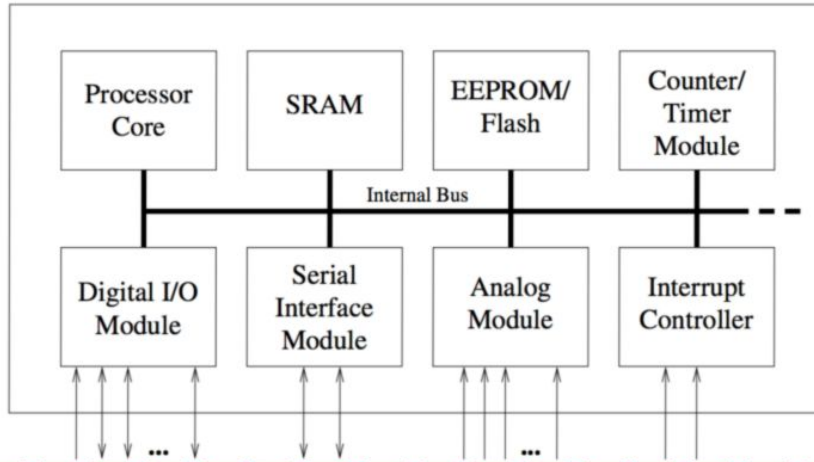
Embedded Systems in IoT Devices

- **Embedded Systems** makes an Object an IoT Device
- **ES = Microcontroller+Sensors+Actuators**
- **Microcontroller**: the computational element of the Embedded System
- Microcontroller needs to interact with the physical world:
 - **Sensors** (Input: physical world->MC)
 - **Actuators** (Output: MC->physical world)
- Microcontroller has a set of Input/Output Pins to interact



Input/Output Pins

Microcontroller



All components are connected via an internal bus and are all integrated on one chip. The modules are connected to the outside world via I/O pins.



The program is stored in the **Flash** and executed by the **Processor** using values computed and stored in the **SRAM**

Timer: trigger periodic events on some pins (read or write something)

Serial Interface: for connecting with other devices (e.g., a computer to write the program in the flash)

Interrupt controller: can be programmed to watch when something “happens” in a pin

Digital and Analog modules: they read and write values on pins

Understanding ESP32 Pins



GPIO Multiplexing

Pins are multiplexed and can serve multiple purposes based on software configuration.



Restricted Pins

GPIO6 - GPIO11 are connected to internal flash memory. **Do not use these.**



3.3V Logic Levels

All GPIO pins operate strictly at 3.3V. Ensure peripherals are compatible.



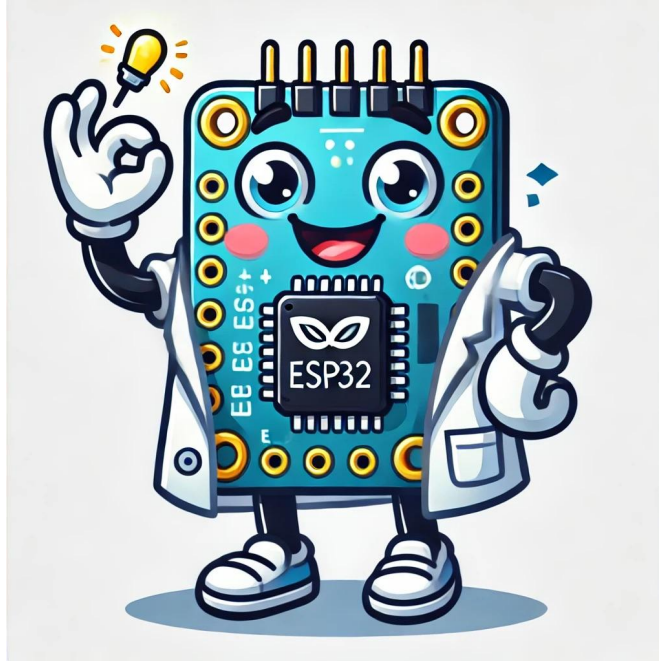
Power Supply (5V)

The board accepts 5V via USB or Vin. An onboard regulator scales this down to 3.3V.

Note: More specific instructions about pins will be provided in the following sections.

ESP32

Basic Reference



Arduino Program Structure

```
#include <Library.h>

// Global variables
int sensorValue = 0;

void setup() {
  // Initialize pins
  pinMode(2, INPUT);
}

void loop() {
  // Main logic
  processData();
  delay(1000);
}
```

1. Setup & Definitions

Import libraries and declare global variables. These are accessible throughout the entire program.

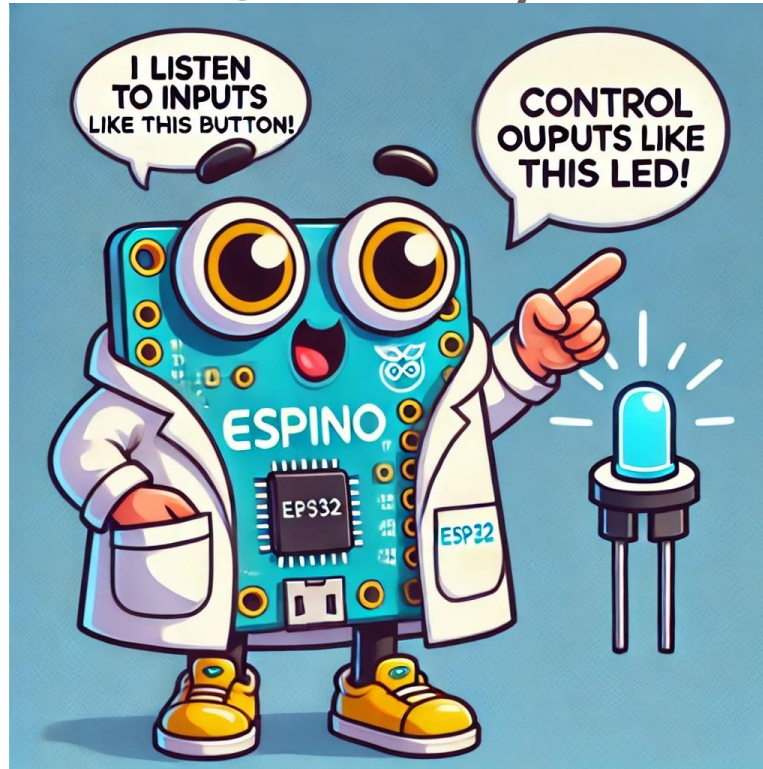
2. void setup()

Runs **once** when the board powers up. Used for initializing pin modes, serial communication, and sensors.

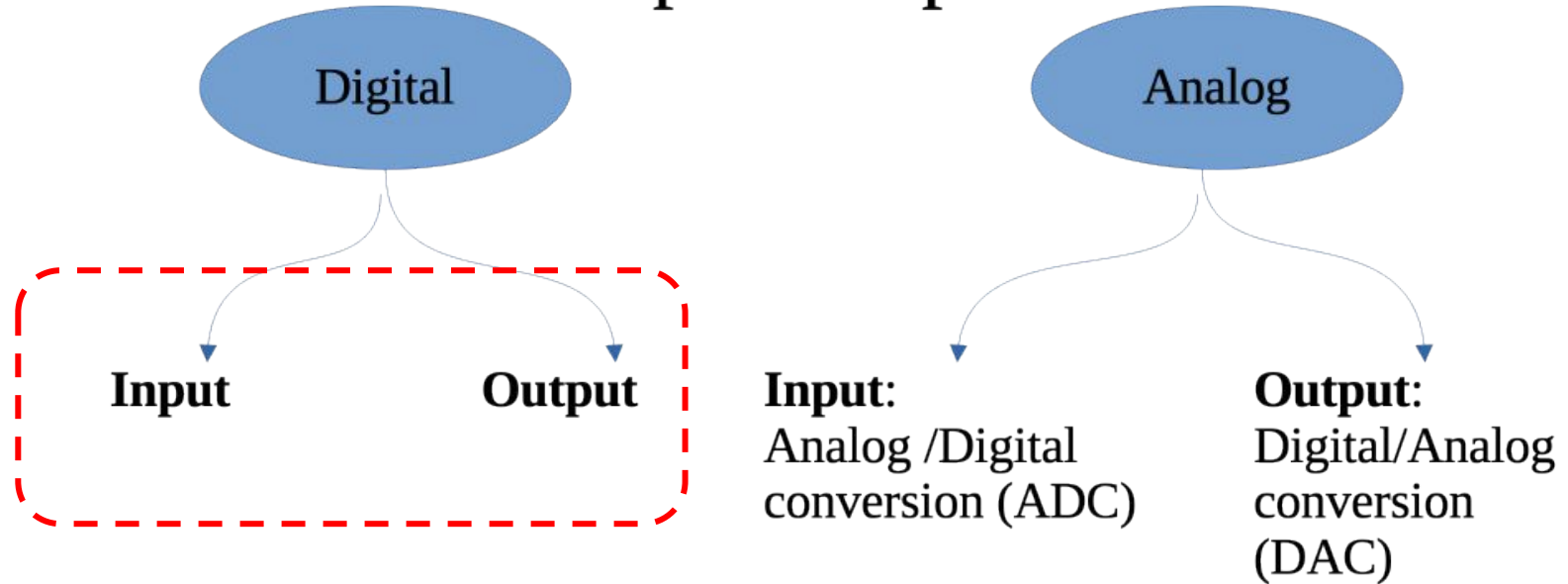
3. void loop()

Runs **continuously** in a cycle. This is where the main logic of your program resides and repeats.

Digital I/O



Input/Output



Input = “to the microcontroller”, output = “from the microcontroller”
Input pins are connected to *sensors*, Output pins to *actuators*

pinMode(pin, mode)

Configures the specified pin to behave either as an **INPUT**, an **OUTPUT**, or an **INPUT_PULLUP**.

INPUT

Reads signals from sensors or switches. The pin is in a high-impedance state.

OUTPUT

Sends signals to devices like LEDs or motors. The pin can provide a substantial amount of current.

INPUT_PULLUP

Activates an internal pull-up resistor. Useful for buttons to avoid external resistors.

```
pinMode(13, OUTPUT); // Sets pin 13 as an output
pinMode(2, INPUT); // Sets pin 2 as an input
pinMode(7, INPUT_PULLUP); // Sets pin 7 as input with pull-up
```

Digital I/O Functions

digitalWrite(pin, value)

Sets a digital pin to **HIGH** or **LOW**.

Syntax:

```
digitalWrite(pin, HIGH/LOW)
```

Use:

Controls devices like LEDs or relays.

digitalRead(pin)

Reads the state of a digital pin as **HIGH** or **LOW**.

Syntax:

```
digitalRead(pin)
```

Use:

Detects input from buttons, switches, or sensors.

```
// Example Usage:
```

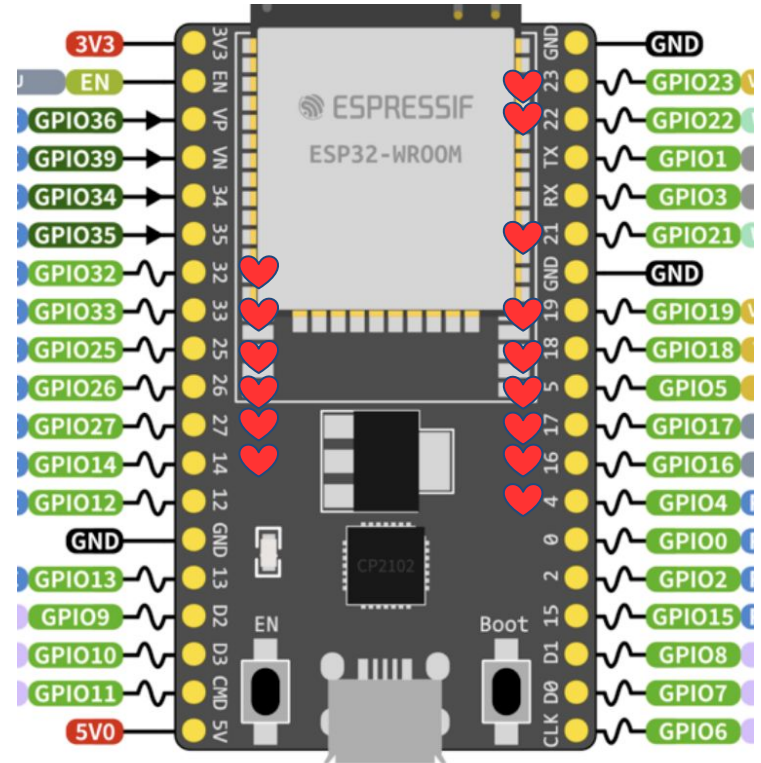
```
digitalWrite(13, HIGH); // Turn on LED on pin 13
```

```
int state = digitalRead(2); // Read button on pin 2
```

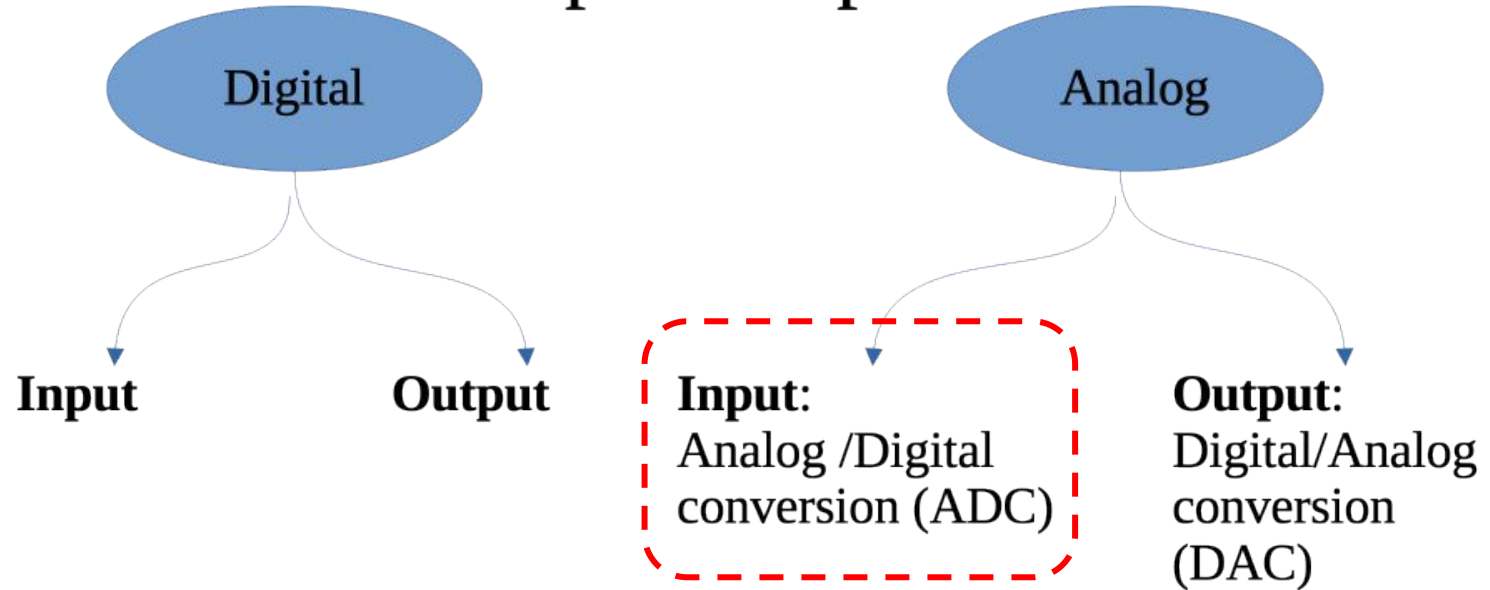
Digital I/O

When sufficient, for digital values these ❤️ are the NO PROBLEM pins

Notice: 34-39 pins are not in the “NO PROBLEM” list because they are digitalRead() input ONLY, but you can use them if you need more inputs



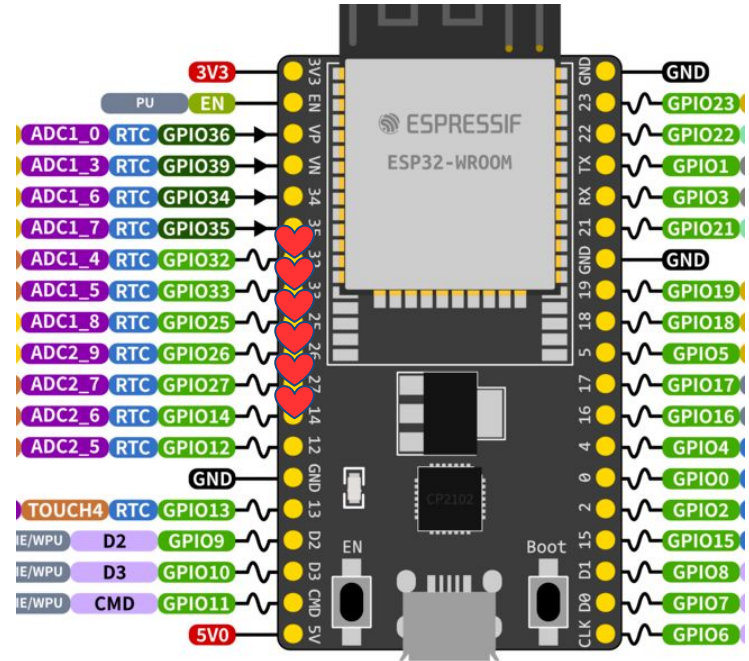
Input/Output



Input = “to the microcontroller”, output = “from the microcontroller”
Input pins are connected to *sensors*, Output pins to *actuators*

Analog Input (ADC)

- Use pins 32-39 to avoid problems with wifi interference
- Analog voltage values between 0 and 3.3v are converted to 12 bits digital values



analogRead(pin)

Reads the value from an analog pin as a number between **0** and **4095** (on ESP32, depending on resolution).

Syntax

```
analogRead(pin)
```

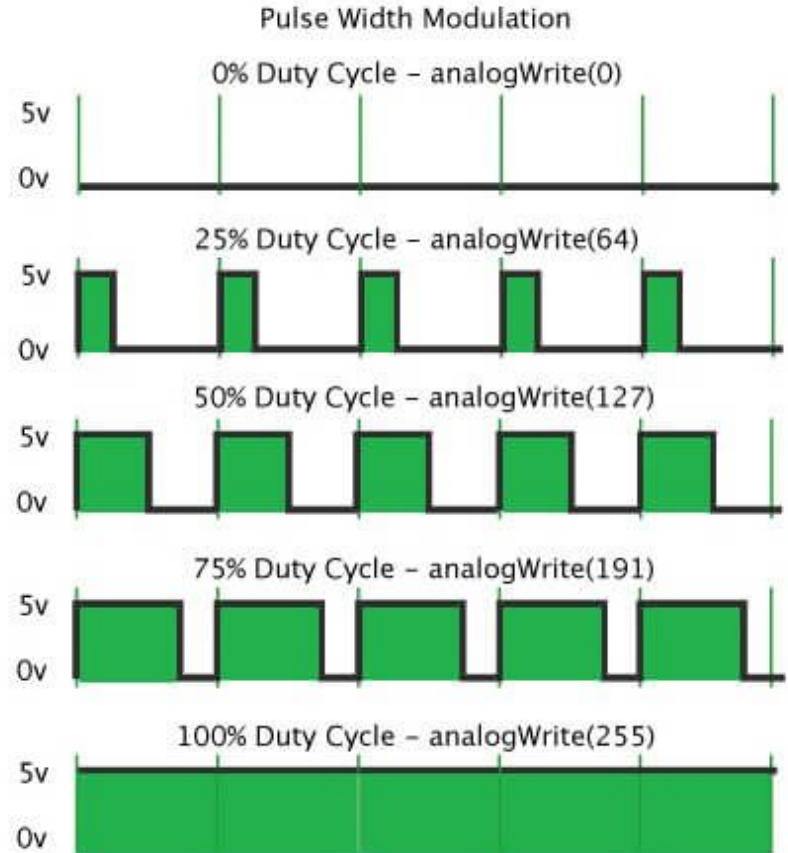
Use

Measures varying voltage levels from sensors like potentiometers or light sensors.

```
// Example Usage:  
int sensorValue = analogRead(34); // Reads analog value from pin 34
```

analogWrite(pin, value)

- Outputs a simulated analog signal using **Pulse Width Modulation (PWM)**.
- **Syntax:** `analogWrite(pin, value)`
 - **value:** A number between **0** (0% duty cycle, off) and **255** (100% duty cycle, fully on).
- **Use:** Controls brightness of LEDs, speed of motors, etc.
- **Example:**
`analogWrite(13, 128); // Sets pin 13 to 50% brightness`



Advanced PWM Control: ledcWrite()

Advanced mode for controlling PWM. Before, configure the PWM with:

```
bool ledcAttach(uint8_t pin, uint32_t freq, uint8_t resolution);
```

Parameters:

- pin: The GPIO pin number to which the PWM signal will be attached.
- freq: The desired frequency of the PWM, realistically 1000-5000 Hz)
- resolution: The resolution of the PWM signal in bits (typically 8,10, 13 bit).

```
const uint8_t pwmPin = 13;      // GPIO pin for PWM output
const uint32_t pwmFreq = 5000;  // PWM frequency: 5 kHz
const uint8_t pwmResolution = 8; // PWM resolution: 8-bit

void setup() {
    // Attach the pin to the LEDC peripheral with the specified frequency and resolution
    bool success = ledcAttach(pwmPin, pwmFreq, pwmResolution);
    if (!success) {
        // Handle error: configuration failed
        Serial.begin(115200);
        Serial.println("Error: LEDC configuration failed!");
        while (true); // Halt execution
    }
}

void loop() {
    // Set the PWM duty cycle (e.g., 50% duty cycle)
    uint32_t dutyCycle = 128; // For 8-bit resolution: 128/255 ≈ 50%
    ledcWrite(pwmPin, dutyCycle);
    delay(1000);
}
```

PWM Considerations: Frequency & Resolution

ESP32's PWM operates using a fixed internal clock frequency, creating a trade-off between speed and precision.

High Frequency Low Resolution

- Higher frequencies of PWM mean shorter periods.
- Fewer clock cycles can "fit" inside each period, limiting the number of available steps (granularity).

High Resolution Max Freq Limit

- Fixing resolution (e.g., 8-bit / 256 levels) requires a minimum number of clock cycles per period.
- This imposes a physical cap on how fast the PWM can cycle.

Typical Configuration Values

LEDs

5 kHz, 8-bit

Balance of smooth fading and high speed.

Motors

1 kHz, 10-bit

Provides finer granularity for precise speed control.

Audio / Buzzers

5-10 kHz, 8-bit

Optimized for generating audible frequency ranges.

dacWrite() Analog Output

True Analog Output: Only pins 25 and 26 can output a continuous voltage between **0V** and **3.3V** directly, without PWM or filtering.

Function Syntax

```
dacWrite(pin, value);
```

pin: GPIO 25 or GPIO 26

value: 8-bit (0-255) for 0V to 3.3V

Code Example

```
dacWrite(25, 128);  
  
// Outputs ~1.65V (50% of 3.3V)  
// on GPIO 25
```

Voltage Mapping

0 (0V) <----- 128 (1.65V) -----> 255 (3.3V)

Feature	DAC (GPIO 25/26)	PWM (<code>ledcWrite</code>)
Output Type	True analog voltage (0-3.3V)	Simulated analog via PWM
Pins	Only GPIO 25, GPIO 26	Any GPIO
Resolution	8-bit (0-255)	Up to 20-bit (1-1048575 duty cycle)
Frequency Control	Not applicable	Configurable (1 Hz to ~40 MHz)
Filtering Required	No	Yes (for smooth analog output)

When to Use DAC vs. PWM:

- **Use DAC (GPIO 25/26):**
 - When precise, smooth analog output is needed directly from the microcontroller (e.g., audio signals, simple analog waveforms).
- **Use PWM (`ledcWrite`):**
 - When you need analog-like control (e.g., LED brightness, motor speed) on any GPIO pin.
 - When higher resolution or frequency is required for specific applications.

tone() Function Guide

PWM Frequency Control: Generates a square wave of a specified **frequency** (50% duty cycle) on a pin, commonly used for buzzers or speakers.

Syntax

```
tone(pin, frequency)
```

Starts the tone indefinitely.

```
tone(pin, freq, duration)
```

Plays for a set duration (ms).

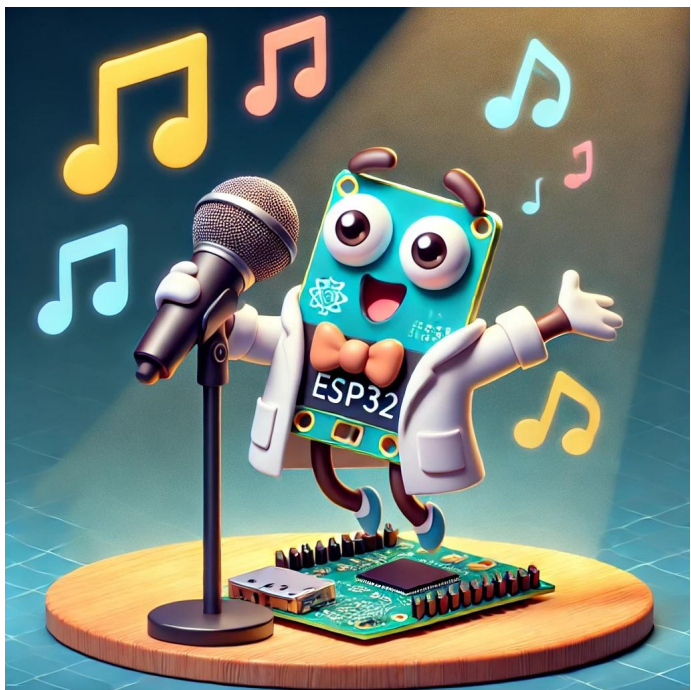
Code Example

```
tone(13, 1000, 500);
```

```
// Plays a 1 kHz tone on pin 13  
// for 500 milliseconds
```

Important Notes:

- Only one `tone()` can play at a time across all pins.
- Use `noTone(pin)` to stop; for multiple simultaneous PWM signals, use `ledcWrite` instead.



Musical Notes / Frequencies

Note	Frequency (Hz)
C ₄	261
C [#] ₄ /D ^b ₄	277
D ₄	293
D [#] ₄ /E ^b ₄	311
E ₄	329
F ₄	349
F [#] ₄ /G ^b ₄	369
G ₄	392
G [#] ₄ /A ^b ₄	415
A ₄	440
A [#] ₄ /B ^b ₄	466
B ₄	493

Note	Frequency (Hz)
C ₅	523
C [#] ₅ /D ^b ₅	554
D ₅	587
D [#] ₅ /E ^b ₅	622
E ₅	659
F ₅	698
F [#] ₅ /G ^b ₅	739
G ₅	783
G [#] ₅ /A ^b ₅	830
A ₅	880
A [#] ₅ /B ^b ₅	932
B ₅	987

delay(milliseconds)

- Pauses the program for the specified **number of milliseconds**.
- **Syntax:** `delay(ms)`
- **Use:** Creates a blocking delay, useful for simple timing tasks.

Example:

```
digitalWrite(13, HIGH); // Turn on LED  
delay(1000);           // Wait for 1 second  
digitalWrite(13, LOW);  // Turn off LED
```

Note: Avoid excessive use in complex programs, as it halts all other code execution. Use **non-blocking techniques** (e.g., `millis()`) for multitasking.



millis()

- **Returns:** The number of milliseconds since the program started running (unsigned long).
- **Syntax:** `millis()`
- **Use:** Tracks time without blocking code execution, ideal for multitasking and timing events.

Note: `millis()` rolls over after ~50 days (unsigned 32-bit integer), so consider overflow handling if running long-term applications.

```
unsigned long previousMillis = 0; // Stores the last time an action was performed
const long interval = 1000;      // 1-second interval

void loop() {
    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis; // Update time
        digitalWrite(13, !digitalRead(13)); // Toggle LED
    }
}
```

ESP32 Serial Communication

Serial Instance

A predefined instance of `HardwareSerial` on UART0, used for USB debugging and programming.

Functions

- `.begin(rate)`: Init with baud rate (e.g. 115200)
- `.println(data)`: Print data with a newline

CODE EXAMPLE

```
void setup() {  
    Serial.begin(115200); // Start Serial  
}  
  
void loop() {  
    Serial.println("ESP32 is running!");  
    delay(1000); // Wait 1 second  
}
```

HardwareSerial Class

Definition

`HardwareSerial` is the class used to control additional UART interfaces (UART1 and UART2) for communication with multiple external devices.

Key Advantage

Flexibility: Choose between UART 1 or 2 and map them to almost any GPIO pins, unlike the default `Serial` instance.

PRACTICAL IMPLEMENTATION EXAMPLE

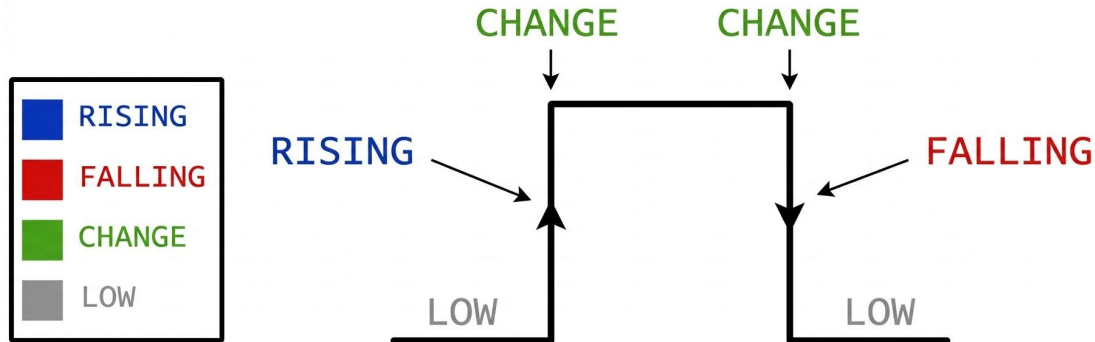
```
HardwareSerial MySerial(1); // Initialize UART1

void setup() {
    MySerial.begin(9600, SERIAL_8N1, 16, 17); // RX=16, TX=17
    MySerial.println("Hello from UART1!");
}
```

attachInterrupt(pin, ISR, mode)

Purpose: Attaches an interrupt to a GPIO pin, allowing the ESP32 to execute a specific function when an event occurs.

- **pin:** GPIO pin number (almost any, except for 6-11)
- **ISR:** Interrupt Service Routine (function to execute).
- **mode:** Type of event to trigger the interrupt:



- Use the **IRAM_ATTR** keyword for ISR functions to ensure they execute from RAM, not flash
- Keep ISR functions **short and fast**, avoiding delays or heavy processing.
- Detach with **detachInterrupt(pin)** when no longer needed.

Be careful about pins

General GPIOs	Most GPIOs support interrupts.
Input-Only Pins	GPIOs 34-39 (input-only, interrupt supported).
SPI Flash Pins	GPIOs 6-11 (cannot be used).
Strapping Pins	GPIOs 0, 2, 12, 15 (use with caution).

```
void IRAM_ATTR handleInterrupt() {
    Serial.println("Button pressed!");
}

void setup() {
    Serial.begin(115200);
    pinMode(27, INPUT_PULLUP); // Button connected to GPIO 27 with pull-up
    attachInterrupt(27, handleInterrupt, FALLING); // Trigger on button press
}

void loop() {
    // Main loop can perform other tasks
}
```

ESP32 Memory

Region	Technology	Purpose	Notes
IRAM	SRAM	Stores executable code (ISRs, etc.).	Faster, directly accessed by CPU.
DRAM	SRAM	Stores runtime data (variables, stack).	Also called Data RAM.
Flash	Non-volatile memory	Stores the program and static data.	Slower than SRAM.
RTC Memory	SRAM	Stores data during sleep modes.	Special-purpose RAM.

Notice: DRAM is not “dynamic RAM”, in this context DRAM denotes “data RAM”